

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)
ORGANISATION OF ISLAMIC COOPERATION (OIC)
Department of Computer Science and Engineering (CSE)

SEMESTER FINAL EXAMINATION
DURATION: 3 HOURS

WINTER SEMESTER, 2024-2025
FULL MARKS: 150

CSE 4539: Web Programming

Programmable calculators are not allowed. Do not write anything on the question paper.

Answer all 6 (six) questions. Figures in the right margin indicate full marks of questions with corresponding COs and POs in parentheses.

1. a) A university launches a new Digital Campus Portal that students use to access course materials, exam results, and the research lab booking system. The faculties can input marks for their assigned courses and the admin can assign courses to the faculties and keep track of financial records.

3 +
 3 + 4
 (CO1)
 (PO1)

When Nadia, a computer science student, tries to log in from her laptop, the portal first asks for her username and password. Nadia types the password that only she remembers. After entering the correct password, the system sends a six digit code to her smartphone through a university authentication app. Nadia opens the app and enters the code into the portal. Once the code is verified, the system asks Nadia to scan her fingerprint on the laptop sensor because she is accessing sensitive exam records. After the fingerprint is verified, the portal confirms that Nadias identity is valid and logs her into the system.

After logging in, Nadia can see her own course materials, assignment submissions, and grades. However, she cannot open the faculty grading dashboard or the administrative financial records. Those sections are only available to professors and university administrators. Later, Nadia goes to the AI research lab booking page. Since she is registered as a member of the AI research group, the system allows her to reserve lab equipment. Another student who is not part of the research group can log into the portal but cannot access the lab booking system.

 - i. In the context of authentication, identify an example of something you know, something you have and something you are.
 - ii. Which parts of the scenario demonstrate two factor and multi factor authentication? Explain your answer.
 - iii. What actions in the scenario represents authentication and authorization?
- b) With proper example explain any 3 types of middlewares.

3 × 5
 (CO1)
 (PO1)
2. In each of the following scenarios, identify the most appropriate React Hook to use. Briefly explain your choice and provide a sample code snippet showing how the hook would be implemented.

5 × 5
 (CO2)
 (PO2)

 - a) A university portal's Course List page displays courses for the current semester. On load, the frontend fetches course data from the backend and displays the list. When the user changes the selected semester via a dropdown, the application fetches the updated course list again.
 - b) Several pages (Users, Products, Orders) repeat the same data-fetching logic: initialize loading state, fetch data from an API, handle errors, store the response, and stop loading. The goal is to improve code reuse, readability, and maintainability.
 - c) A feedback form automatically places the cursor in the message input field on page load. Upon submission, the field is cleared and focused again, allowing the user to quickly type another message.

d) After login, the application stores user information (username, role, authentication token). Many components need access: Navbar shows the username, Dashboard displays the role, Profile page shows account details, and some components check authentication.

e) A dashboard displays hundreds of StudentRow components, each with a Delete button. The parent passes a handleDeleteStudent function to every row. On parent re-render, the function is recreated, causing all rows to re-render unnecessarily. To fix this, the delete function should be memoized so its reference remains stable unless dependencies change.

3. You already have a simple backend system for No Pain No Lie Gym, where honesty is mandatory. Each gym member has basic information such as id, name, age, email, contact, weight and membershipType.

The backend system supports the following features. A new member can be saved only if their age is greater than 18. Anyone younger is politely rejected with a suggestion to join the Yoga for Toddlers program. The system can sort members by age (youngest to oldest) to decide who gets cardio and who gets light stretching. It can also retrieve a members record using a specific member ID. The system supports targeted motivational campaigns based on membership type and weight range. Finally, if a members information changes (for example, weight, membershipType, or contact), only the modified fields are updated while the rest remain unchanged.

You are now required to design and implement the frontend application for the same system. Build a simple user interface that communicates with the previously implemented backend APIs. Your frontend should allow users to perform the following actions:

a) **Add a New Gym Member**

Create a form where users can enter the following information: name, age, contact, email, membership type, and weight. The gym has three membership types: General, Premium, and Elite. These options must be displayed using a dropdown list. All fields are mandatory and cannot be left empty.

When the form is submitted, the frontend must first perform client-side validation to ensure that all required fields are filled and the data types are appropriate (for example, age and weight must be numeric). If the validation passes, the frontend should send a request to the backend API to create the new member and then display the response message returned by the server.

Consequently, the newly added user should be displayed in the list below along with all other users. For now, display only the users name, age, and membership type.

b) **View Members Sorted by Age**

Provide a button that sends a request to the backend API to retrieve the list of members sorted from youngest to oldest. The retrieved members should be displayed clearly on the page.

c) **View Target Campaign Members**

Provide input fields where the user can select a membership type and specify a weight range (minimum and maximum weight). After submitting the request, the frontend should retrieve and display the list of members that match the specified criteria.

d) **Update Member Information**

Create a form that allows updating one or more fields of a members record (for example, weight, membership type, or contact). The user must provide the member ID along with the fields that need to be updated. The frontend should send only the modified fields to the backend API so that the remaining information stays unchanged.

12
(CO2)
(PO2)

5
(CO2)
(PO2)

5
(CO2)
(PO2)

8
(CO2)
(PO2)

4. In continuation of Question 3, the gym now also maintains user accounts for three roles: `member`, `trainer`, and `admin`. Each role has a `username` and `password` stored in the database. `Passport.js` is already configured with the Local Strategy for `username/password` authentication. You do not need to modify or create any `Passport` configuration files.

When a user logs into the gym portal, the system should authenticate the user using `username` and `password`. After successful authentication, the system must determine what resources the user is allowed to access based on their role (`member`, `trainer`, or `admin`).

- a) You are tasked to build the backend of the gym portal using `Node.js`, `Express`, and `Passport.js` to enforce role-based access control.

5 × 3
(CO3)
(PO2)

- i. Create a login API that authenticates a user using `Passport.js` with `username` and `password`.
- ii. Create protected routes for members, trainers, and administrators separately that allow only authenticated users with their respective role to access their resources.
- iii. Implement authorization middleware that checks the role of the logged-in user before granting access to protected routes.
- iv. Ensure that if a member attempts to access the trainers workout management page or the admins financial records, the system denies access and returns an appropriate response message.
- v. If a user tries to access any protected resource without logging in, the system should return a response indicating that authentication is required.

- b) In the context of Question 4.a, design a simple frontend interface that interacts with this backend system. The frontend should allow users to log in and access different sections of the portal based on their role.

15
(CO3)
(PO2)

Create a login page that contains a field for `username`, a field for `password`, and a login button. When the user submits the form, the frontend should send a request to the backend login route. If authentication is successful, the user should be redirected to their protected pages based on their role. For example:

- Members should be redirected to the Member Dashboard (where they can view and update their own profile).
- Trainers should be redirected to the Trainer Workload Manager (where they can view all members and assign workouts).
- Administrators should be redirected to the Admin Panel. There will be two links or buttons: Manage Memberships and Financial Records.

If the backend returns an unauthorized or forbidden response, the frontend should display a message such as: Access denied. You are not authorized to view this resource.

5. In continuation of Question 3, every new member must complete a registration process before accessing gym facilities. The gym maintains a digital portal where members can upload documents for verification.

When a new member joins the gym, they must upload a profile photo so that staff can identify them at check-in. Since an official photo is required for gym access, the system only accepts one image file for this purpose.

Once registered, members can submit progress reports of their fitness journey. These reports must include evidence of their workouts, such as recordings of weightlifting sessions, treadmill runs, or group class participation. Because a single progress report may involve several workout moments worth documenting, members are allowed to upload multiple media files upto 10, which may include images, audio recordings or video clips. These files help the trainers verify the authenticity of the fitness claims.

The backend system for the gym portal has already been implemented and provides the necessary API endpoints for uploading images and media files. Your task is to design the frontend interface that allows members to interact with the system and upload the required files.

- a) Design a frontend component that allows a new member to upload their profile photo during registration. The interface should allow the user to select only one image file, preview the selected image before submission, and then send the image to the backend API [POST /api/members/profile-image] when the form is submitted. 6
(CO2)
(PO2)
 - b) Design another frontend component that allows members to submit workout evidence. In this case, the user should be able to select multiple files at once, but the system should accept only audio and video formats. When files are selected, the frontend should display the names of the selected files before submission and then send them to the backend API [POST /api/progress/evidence] in a single request. 7
(CO2)
(PO2)
 - c) After the files are successfully uploaded, the system should display the uploaded evidence so that the member can confirm that their submission was successful. Describe how the frontend would display audio files using an audio player and video files using a video player. Provide a short code example demonstrating how one uploaded media file would be rendered on the page. API: [GET /api/progress/:progressId/evidence] 7
(CO2)
(PO2)
6. a) Explain each parts of React component life-cycle. 12 + 8
b) With proper example explain why do we need props.children. (CO1)
(PO1)